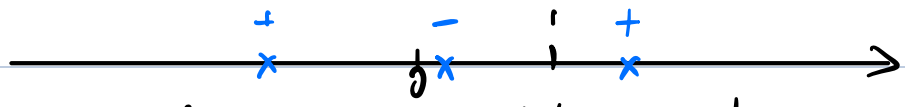
Non-linear Classification



Not separatable by linear classifier

feature transformation

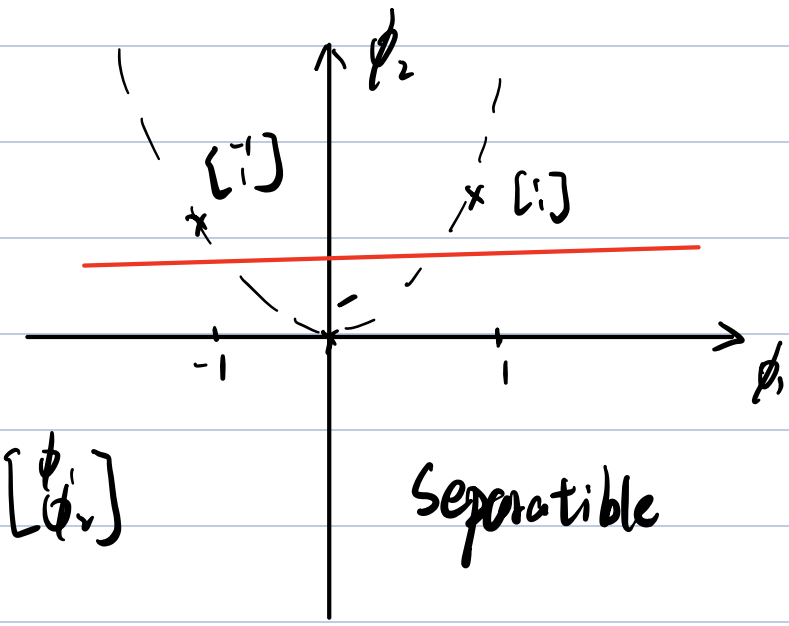
$x \to \phi(x) = \begin{bmatrix} x \\ x^2 \end{bmatrix}$

$\theta \to \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$

new classifier: $h(x; \theta; \theta_0) = \text{sign}(\theta \cdot \phi(x) + \theta_0)$

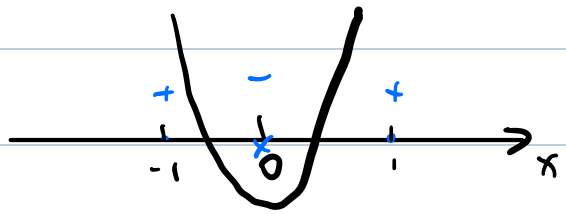
$= \text{sign}(\theta_1 \cdot x + \theta_2 \cdot x^2 + \theta_0)$

In feature space

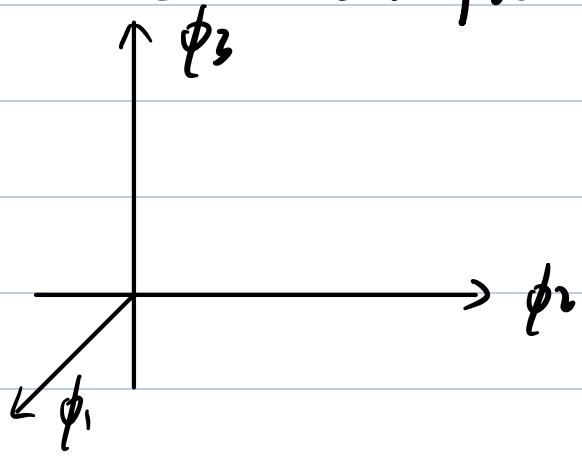


$\phi(x) = \begin{bmatrix} x \\ x^2 \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \end{bmatrix}$

Separatible



3D example



$$\phi(x) = \begin{bmatrix} x_1 \\ x_2 \\ x_1 x_2 \end{bmatrix}$$

Polynomial features

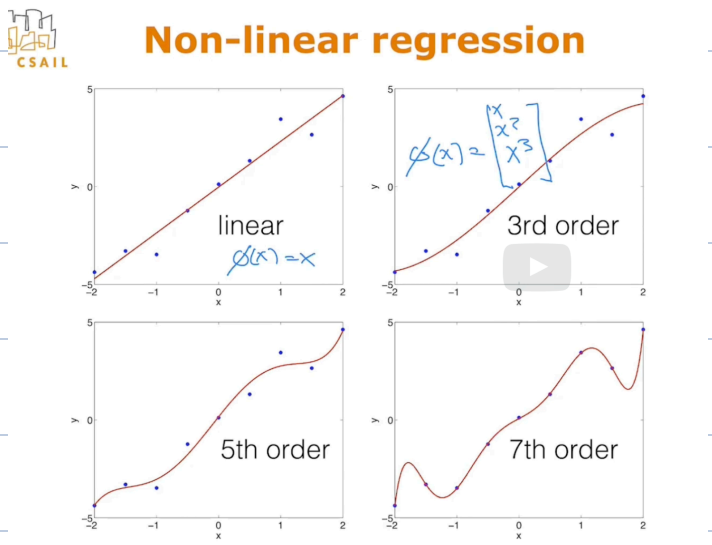
$$x \in \mathbb{R}, \quad \phi(x) = \begin{pmatrix} x \\ x^2 \\ x^3 \\ \vdots \end{pmatrix}$$

e.g

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \in \mathbb{R}^2$$

$$\phi(x) = \begin{pmatrix} x_1 \\ x_2 \\ x_1^2 \\ x_2^2 \\ x_1 x_2 \end{pmatrix} \in \mathbb{R}^5$$

Which one to choose?



use cross validation

缺点: End up very high D feature

e.g. $\phi(x) = [x_1, \ldots\ldots, x_d, \sum x_i x_j, \sum x_i x_j x_k, \ldots]$
$x \in d$

Inner products, kernels

Compute the inner product between 2 feature vectors can be cheap.

$\phi(x) = [x_1, x_2, x_1^2, \sqrt{2} x_1 x_2, x_2^2]^T$

$\phi(x') = [x_1', x_2', x_1'^2, \sqrt{2} x_1' x_2', x_2'^2]^T$

$$\phi(x)\cdot\phi(x') = (x\cdot x') + (x\cdot x')^2$$

The inner product of 2 feature vectors can be evaluated cheaply based on original examples.

Kernal function: A function of 2 examples correspond to an inner product between some feature vectors.

$$\ker(x, x') = \phi(x)\cdot\phi(x')$$

$$x_1^2 + (x_2+x_3)^2$$
$$= x_1^2 + x_2^2 + x_3^2 + 2x_2x_3$$

Kernel Perceptron Algorithm

$$\theta = \sum_{j=1}^{n} \underbrace{\alpha_j}_{\text{number of mistakes}} \cdot y^{(j)}\cdot\phi(x^{(j)})$$

$$\theta\cdot\phi(x^{(i)}) = \left(\sum_{j=1}^{n}\alpha_j y^{(j)}\phi(x^{(j)})\right)\cdot\phi(x^{(i)})$$
$$= \sum_{j=1}^{n}\alpha_j y^{(j)} k(x^{(j)}, x^{(i)})$$

update: previous: $y^{(i)}\cdot\theta\cdot\phi(x^{(i)}) \le 0$

now: $y^{(i)}\sum_{j=1}^{n}\alpha_j y^{(j)} k(x^{(j)}, x^{(i)}) \le 0$

$\alpha_j \leftarrow \alpha_j + 1$

Constructing kernels.

Composition rules.

1. $\ker(x, x') = 1$ is a kernel function
2. let $f: \mathbb{R}^d \to \mathbb{R}$ and $\ker(x, x')$ is a kernel.

Then so is $\tilde{k}(x, x') = f(x) \cdot k(x, x') \cdot f(x')$

3. If $k_1(x, x')$ and $k_2(x, x')$ are kernels, then $k(x, x') = k_1(x, x') + k_2(x, x')$ is also a kernel.
4. If $k_1(x, x')$ and $k_2(x, x')$ are kernels, then $k(x, x') = k_1(x, x') \cdot k_2(x, x')$ is a kernel.

$$\tilde{k}(x, x') = f(x) \cdot k(x, x') \cdot f(x')$$
$$k(x, x') = \phi(x) \cdot \phi(x')$$
$$\tilde{k}(x, x') = f(x) \cdot \phi(x) \cdot \phi(x') \cdot f(x')$$

Radial basis kernel

$$\ker(x, x') = \exp\left(-\frac{1}{2}|x - x'|^2\right)$$

corresponds to infinite dimensional feature vectors.

For perceptron algorithm, we didn't know when to stop the algorithm. But using radial basis kernel,

it will always end in a finite case.